ChatGPT

MemoVerse：Web + Android 数字纪念品收藏与离线近场分享项目技术与产品深度研究

逻辑链：生活票据/照片 → OCR 与语义结构化 → Subject / Experience 分离 → 生成可收藏 Snapshot → 本地优先保存 → 近场 P2P 发现 → 自定义应用层协议可靠复制 → 接收方形成独立 CollectionEntry → 恢复互联网后幂等同步 → Web/Android 状态收敛 → 3D 与 AI 作为视觉增强 → 用“真实网络问题的解决深度”而不是“技术名词数量”支撑 ⩾1.30 难度。

> **研究基线：2026-10-13。**
>
> 本报告默认 **Android + Web 是一个月内必须完成的平台，iOS 不属于 MVP**；四名开发者采用快速迭代、垂直切片而非严格瀑布式分工。老师的 1.30 难度标准没有可量化公式，因此下文的“达到/不足”均指**课程答辩中是否具有充分技术论据**，而不是对最终评分的保证。

执行摘要与结论

MemoVerse 的核心产品概念已经可以收敛成一句话：

> **把现实世界中容易消失的“生活凭证与瞬间”——电影票、高铁票、奶茶贴纸、店铺照片、短视频等——转化成带有结构化信息、视觉素材和可选 3D 表达的数字纪念品；它既可以成为自己的生活档案，也可以在没有互联网时像递出一张实体卡片一样直接分享给身边的人。**

这个项目**不是电子票务系统**，也不是 NFT 或票券所有权转移系统。分享行为是 **Copy / Recommendation**，而不是 Ownership Transfer。这一定位非常重要，因为它直接消除了防伪、双花、所有权共识等大量无关复杂度，同时让“离线 P2P 分享”成为自然的产品功能，而不是为了网络编程课程硬塞进去的技术。

最重要的研究结论

第一，Android 无互联网近场交换完全可行。 Android 官方 Wi-Fi Direct API 支持设备在不依赖传统接入点的情况下发现并建立 P2P 连接；连接建立后，官方文档直接使用 `ServerSocket` / `Socket` 进行字节流传输，因此“Wi-Fi Direct + TCP Socket + 自定义协议”是技术上成立的课程方案。[1]

Google Nearby Connections 则进一步提供完全离线的附近设备通信抽象，会在 Bluetooth、BLE、Wi-Fi 等底层链路间工作，并原生支持 Bytes、File、Stream payload、传输进度、取消和失败回调；它同时有 Android 和 Swift/iOS SDK。它非常适合作为**低风险实现方案或未来跨平台适配器**。[2]

但如果本项目的课程目标是“让老师看见网络编程深度”，我建议：

> **主评分路径：Wi-Fi Direct + 原生 TCP Socket + 你们自己的 MemoShare Protocol。**
>
> **风险后备路径：Nearby Connections。**

不要一开始同时实现两套。先在前 48 小时做实机 Spike；Wi-Fi Direct 在目标设备上稳定，就继续原生路径；如果设备碎片化、权限或连接稳定性明显拖垮进度，再切 Nearby。这样兼顾难度与交付安全。

第二，NAT 穿透不属于 MVP 必要技术。 两台设备通过 Wi-Fi Direct、Nearby 或同一局域网直接通信时，并不存在“公网两端都藏在 NAT 后面，需要穿透”的问题。ICE 是用于 NAT Traversal 的协议体系，会利用 STUN 与 TURN；应把它写在“未来跨公网 P2P 扩展”，而不是为了 1.30 强行实现。 [3]

第三，1.30 的真正论据不是 OCR、Meshy、COS 的数量，而是四层网络问题同时落地：

难度证据	最低应完成的内容	课程价值判断
P2P Connectivity	无互联网发现、连接、角色协商	**直接网络编程**
Application Protocol	HELLO → OFFER → ACCEPT → MANIFEST → TRANSFER → VERIFY → COMMIT	**直接网络编程，强证据**
Reliability	中断恢复、完整性检查、重复导入幂等、原子 Commit	**强证据**
Distributed Sync	Offline operation queue、revision、conflict、tombstone、最终收敛	**强证据**
Cloud Architecture	API、PostgreSQL、对象存储、Worker	中高工程难度
OCR / Edge AI	ML Kit、本地/云端识别、人工纠错	非网络难度补充
3D Pipeline	AnyImage/TripoSR/Meshy、异步任务、资产归档	非网络难度补充
Capacitor / Native Bridge	Web UI 与 Kotlin 系统能力桥接	工程难度补充

如果最终只有“React + OCR API + Meshy API + COS + Nearby SDK 传文件”，技术栈看起来很多，但**网络课程论证仍然可能偏弱**。相反，只要你们现场演示“断网 → 两机发现 → 自定义协议 → 中途断开 → 恢复 → 验证 → 导入 → 重新联网 → 云端同步 → 冲突正确处理”，1.30 的论据会非常扎实。

第四，AnyImage 可以部署在服务器或个人电脑 Worker，但“一秒生成”目前不能当成已验证事实。
AnyImage 官方仓库把它描述为 Blender 的 image-guided structure building 工具，可以基于图片建立带材料、轮廓、深度等特征的场景元素；它更接近“快速空间化/立体化图片”的工具，而不是完整 AI 单图 3D 重建模型。其仓库使用 GPL-3.0-or-later。 [4]

这实际上很符合 MemoVerse：一张电影票或奶茶贴纸未必需要生成高度准确的完整 3D 物体，做成一个有厚度、纹理、浮雕和景深感的“数字纪念品”反而更符合产品审美。

但我没有找到 AnyImage 上游提供的可靠“一秒”性能基准。因此：

> **一秒应该定义成你们的实验目标，而不是开题报告里的既成指标。**

作为对照，TripoSR 官方仓库给出了在 NVIDIA A100 上单图重建低于约 0.5 秒的基准，并支持纹理烘焙，MIT License；这证明“单图快速 3D”本身可以达到亚秒级，但不能推导出 AnyImage 在你们电脑或云服务器上也一定一秒完成。 [5]

第五，产品并不是完全空白市场。 PocketStubs 已支持扫描实体电影票并自动加入观影记录，还具有社交和 AI 艺术功能；TicketShelf 已覆盖演唱会、体育、展览、电影、旅行等票根，支持 AI 辅助扫描、照片、周边、地图、年度回顾和共享经历。因此，“OCR 扫票根 + 数字收藏”本身不能再作为核心创新声明。 [6]

MemoVerse 最值得建立的差异化是：

Memorabilia，而不是 Ticket。

Local-first，而不是 Cloud-only。

Nearby handoff，而不是普通分享链接。

Recommendation copy，而不是虚构的所有权交换。

2D/3D visual artifact，而不是数据库记录。

Received recommendation → later experience 的记忆链，而不是单一票根列表。

这个方向有明显的产品空间。

近场通信与跨平台技术可行性

Android 离线分享是整个项目最值得优先验证的技术，因为它既决定产品体验，也决定网络课程难度。

四种近场方案的实用比较

方案	无互联网	发现方式	数据链路	大文件能力	Android	iOS / 跨平台	开发难度	对 1.30 的价值
Wi-Fi Direct + TCP	✅	Wi-Fi P2P Discovery	自己的 TCP Socket	高，适合图像/视频/GLB	✅ 官方 API	Android API 无法直接复用于 iOS	**高**	**最高**
Nearby Connections	✅	SDK 自动管理附近发现	Bluetooth/BLE/Wi-Fi 抽象	官方支持 File/Stream	✅	✅ 有 Swift/iOS SDK	中	中高
Bluetooth Classic	✅	蓝牙发现/配对	RFCOMM / BluetoothSocket	可以，但更适合较轻数据	✅	不能假设 Android 实现直接跨 iOS	中	中
Local Hotspot + TCP	✅	用户进入同一局域网	普通 LAN TCP	高	可做	理论上可用普通 IP 网络模型	中	中
Internet/WebSocket	❌	云账号/服务器	TCP/TLS	高	✅	✅	低	低

方案	无互联网	发现方式	数据链路	大文件能力	Android	iOS / 跨平台	开发难度	对 1.30 的价值
ICE/STUN/TURN	需要某种公网协调基础设施	Candidate discovery	P2P / Relay	高	可实现	可实现	很高	对当前需求**过度设计**

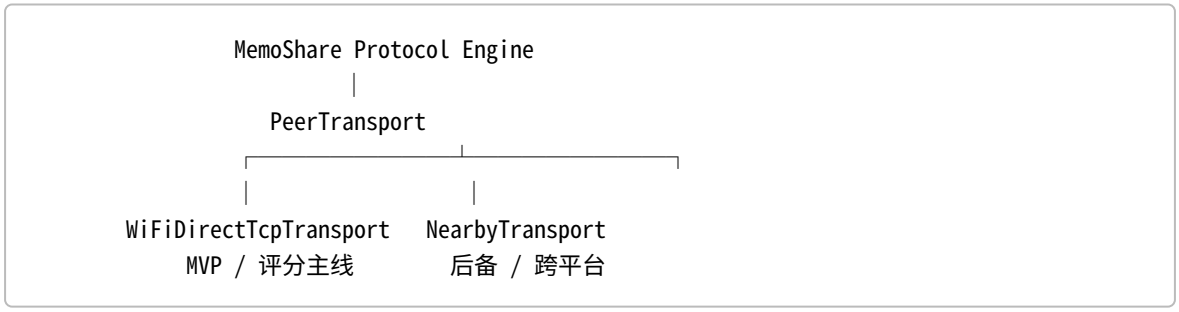
Android 官方说明 Wi-Fi Direct 可以让支持的设备直接发现并连接而不依赖传统无线接入点，并在连接完成后使用普通 `ServerSocket`、`Socket`、输入输出流通信。Android 13 之后还需要处理 `NEARBY_WIFI_DEVICES` 等权限，旧版本的发现流程涉及位置权限/位置模式，因此**权限与机型差异必须实机测试**。 [1]

Nearby Connections 的优势恰恰相反：它把底层发现与链路选择抽象掉。Google 官方说明其连接可以完全离线工作，会利用 Bluetooth、BLE 与 Wi-Fi，并支持对等连接确认、Bytes、File 和 Stream 类型 payload。Android 端依赖 Google Play services 并需要相应 Bluetooth / Nearby Wi-Fi 权限。 [7]

因此，对于 MemoVerse：

Wi-Fi Direct 是“课程路线”，Nearby 是“产品路线”。

最理想的代码结构不是让业务逻辑依赖 Wi-Fi Direct，而是：



也就是说：

```
interface PeerTransport {
  discover(): AsyncIterable<Peer>;
  connect(peerId: string): Promise<Session>;
  sendControl(frame: Uint8Array): Promise<void>;
  sendAsset(asset: AssetSource): Promise<void>;
```

```
  close(): Promise<void>;
}
```

业务层只认识：

```
HELLO
OFFER
ACCEPT
MANIFEST
TRANSFER
VERIFY
COMMIT
```

而不认识 `WifiP2pManager` 或 Nearby API。

这一个抽象决定，会极大降低将来加 iOS 的成本。

Bluetooth 不建议作为主要媒体传输路径。 Android 确实提供 BluetoothSocket，并把它设计成具有输入输出流的 Socket 式接口；经典蓝牙适合持续通信，而 BLE 更强调低功耗。对你们包含高分辨率图片、短视频乃至 GLB 模型的收藏包来说，用 Bluetooth 做主传输只会增加时间和体验压力。 [8]

Nearby 则可以让底层自行在 Bluetooth/BLE/Wi-Fi 等链路间工作，因此如果后期发现 Wi-Fi Direct 某些机型很难稳定，这是最现实的降级方案。 [9]

iPhone 到底怎么办

这里需要更新一个以前常见但现在已经过时的认知：

Apple 的 Multipeer Connectivity 已经不应该作为新项目唯一长期路线。 Apple 当前文档已经将 Multipeer Connectivity 标为 deprecated，并建议迁移到 Network framework。Apple 的 Network framework 与 Bonjour 服务发现可以承担 Apple 生态中的本地网络发现和连接能力。 [10]

对于 MemoVerse 将来扩展 iOS，有两条合理路线：

```
路线 A:
Android ─┐
         ├── Google Nearby Connections ── 跨平台
iOS ─────┘
```

Nearby 官方提供 Swift 接入，iOS 侧要配置 Wi-Fi Information、Hotspot Configuration、Bluetooth、Local Network、Bonjour 等相应权限/Capability，所以**“有 SDK”不等于“一写就百分百跨任意手机运行”**，仍然必须实际用目标 iPhone 与 Android 设备验证。 [11]

或者：

```
Android
  Wi-Fi Direct / Nearby
```

```
iOS
  Network.framework + Bonjour

        ↓
共同实现 MemoShare Protocol
```

第二种架构的价值在于，**平台发现技术可以不同，应用协议和收藏包格式可以相同。**

当前一个月项目不建议做 iOS。把 `PeerTransport` 抽象设计正确即可。

NAT 穿透应该放在哪里

本地 P2P：

```
Android A ←———— Wi-Fi Direct / Nearby ————→ Android B
```

不需要 NAT Traversal。

跨不同网络：

```
校园网 A
   | NAT
   ▼
Internet
   ▲
   | NAT
家庭网 B
```

才进入 ICE/STUN/TURN 问题。

RFC 8445 定义的 ICE 专门用于 NAT Traversal，并结合 STUN 与 TURN 收集和检查候选路径；STUN 可以帮助端点获得 NAT 分配的地址/端口，但 STUN 本身并不是完整 NAT 穿透方案。 [3]

所以本项目可以在报告写：

> Future Work：支持 Remote P2P Sharing，可研究 ICE + STUN + TURN。

不要为了“显得难”在当前一个月里加入。

它解决的是你们目前没有的业务问题。

识别、三维、存储与开源方案

这一层要特别避免一个陷阱：

> **把“调用很多 AI API”误认为“项目架构复杂”。**

正确的设计应该是：基础收藏永远立即可创建；OCR、语义匹配、3D 都是逐步增强。

OCR：在线是主线，离线 ML Kit 是很值得做的增强

Android ML Kit Text Recognition v2 可以直接在设备上提取图像文本，并提供两种模型交付方式：

- **Bundled**：模型在构建时打进 App，包体更大，但无需首次动态下载即可使用；
- **Unbundled**：App 更小，模型通过 Google Play services 动态下载。 12

ML Kit 当前文档列出了 Latin、Chinese、Japanese、Korean 等识别依赖，并明确指出输入图像分辨率、清晰度、对焦质量会影响识别效果。 12

因此建议 MemoVerse 采用：

```
拍照
  │
  ├──── 立即保存原图与基础 CollectionEntry
  │
  ├──── 本地 ML Kit OCR（模型可用则执行）
  │        │
  │        └──── 得到 provisional metadata
  │
  └──── 有网后 Server OCR / Metadata Enrichment
           │
           ├──── 类型识别
           ├──── 字段抽取
           ├──── 内容搜索
           └──── 返回候选
```

注意，是 **candidate**，而不是“AI 自动生成事实”。

例如：

```
OCR:
“万象影城
蜘蛛侠...
18:30
7号厅”
```

系统可以自动建议：

```
{
 "type": "MOVIE",
 "title": "蜘蛛侠：……",
 "venue": "万象影城",
 "time": "18:30"
}
```

然后再搜索海报候选。

但是：

```
一张只有牛排的照片
```

不能凭空可靠推出：

```
店名 = 某某餐厅
菜名 = 澳洲M9和牛
```

因此数据字段必须保留：

```
value
confidence
source
confirmed_by_user
```

例如：

```
{
 "field": "venue_name",
 "value": "万象影城",
 "source": "ocr+parser",
 "confidence": 0.86,
 "confirmed_by_user": false
}
```

这会显著改善后期数据质量。

AnyImage 的真实角色

AnyImage 官方项目定位是 Blender 的快速 image-guided structure building toolkit，可以把图片形成具有材质、轮廓、深度等空间表达的场景元素，并提供诸如 plane、depth plane、cutout、balloon、depth shape 等视觉构造。 13

这意味着它非常适合：

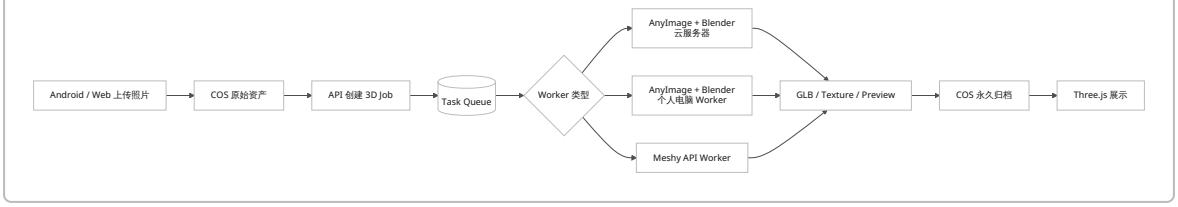
```
电影票根 → 有厚度的票卡
奶茶标签 → 立体贴纸
电影海报 → 浮雕海报
照片 → 带景深层次的立体纪念卡
```

而不是强求：

```
一张电影票
    ↓
准确恢复真实世界中的完整三维物体
```

这比“白模”更符合 MemoVerse。

推荐 Worker 架构：



这里甚至**不一定需要“把公网隧道打到个人电脑”**。

更稳的做法是：

```
你的 PC Worker
     |
     | 主动 HTTPS / WebSocket 出站连接
     ▼
云端 Worker Gateway / Job API
```

由本地 Worker 主动向云端领取任务：

```
GET job
→ 下载 COS 输入
→ Blender 执行
→ 上传 COS
→ POST result
```

这样你的电脑仍然可以充当算力节点，却不用暴露 Blender 服务端口，也不用处理家庭 NAT 入站。

AnyImage 的“一秒目标”怎么科学定义

目前最合理的性能指标应该拆开：

```
T0 用户点击生成

T1 后端接受 Job
T2 Worker 开始
T3 快速 Preview 可见
T4 GLB 生成完成
```

```
T5 上传 COS 完成
T6 Three.js 加载完成
```

“一秒响应”完全可以做到，不代表“一秒完整生成”。

建议你们真正测：

指标	建议目标	是否必须
API 创建 Job	< 500 ms	是
快速 2.5D Preview	< 1 s 感知目标	是
AnyImage warm worker	**实测决定**	实验
Blender cold start	单独记录	实验
完整 GLB 可浏览	数秒级可接受	否
Meshy	异步任务	不追求秒级

任何开题报告都不应该写“AnyImage 可在服务器 1 秒完成”，因为当前没有找到该项目的官方性能基准。

但有一个非常值得试的开源替代：

方案	类型	优点	限制/风险	License
AnyImage	Blender 图片空间化	非常符合票根/贴纸/海报审美；快速视觉效果	不是完整 AI 3D 重建；自动化、headless 与性能需测试	GPL-3.0-or-later [4]
TripoSR	Single-image → 3D	官方给出 A100 亚秒级重建基准；约 6GB VRAM；可烘焙纹理	目标 GPU 上速度需重新测试	MIT [5]
InstantMesh	Single-image → Mesh	完整开源单图 mesh 路线	CUDA/PyTorch 环境较重	Apache-2.0 [14]
Hunyuan3D-2	Shape + Texture	支持本地 API server、代码和 Blender 集成；质量路线更完整	Shape+Texture 官方建议显存明显更高，部署更重；许可需逐版本复核	自定义/模型许可需核查 [15]
Meshy	SaaS Image-to-3D	集成最快、不维护 GPU 模型	API 成本、服务依赖、异步等待	商业服务 [16]

其中 **TripoSR 非常值得在第一轮实验里和 AnyImage 对比**。

AnyImage：

```
视觉风格更适合“电子纪念品”
```

TripoSR：

```
更像真正 Single Image → Mesh
```

你们不需要一开始选择其中一个永远使用，甚至可以定义：

```
Fast 3D
  ├──── AnyImage visual collectible
  └──── TripoSR local reconstruction

Premium / Detailed 3D
  └──── Meshy API
```

Meshy 应该如何接入

Meshy 当前提供 REST API，可以创建并查询生成任务，因此应按**异步任务**设计，而不是让 Web 请求一直等到生成完成。 17

正确流程是：

```
POST /collectibles/{id}/3d
      ↓
Backend 创建 Job
      ↓
调用 Meshy
      ↓
保存 external_task_id
      ↓
PENDING / PROCESSING
      ↓
轮询或 webhook
      ↓
SUCCEEDED
      ↓
立即下载资产
      ↓
上传到自己的 COS
      ↓
Asset 状态 READY
```

特别重要的是：Meshy 官方关于 MCP/API 生成资产的说明明确提示**下载 URL 会过期，需要自行保存希望长期保留的结果**。因此 MemoVerse 不能把 Meshy 的结果 URL 当数据库里的永久资产。 18

必须做：

```
Meshy temporary URL
        ↓
MemoVerse Worker download
        ↓
COS permanent object
        ↓
Asset.object_key
```

而不是：

```
Asset.url = meshy temporary URL   ❌
```

另外，不要把“Meshy 3D”自动等同于“任意纪念品动态动画”。Meshy 的自动 Rigging 文档明确指出，当前程序化骨骼绑定主要适用于具有清晰四肢结构的标准双足人形，对非人形资产并不适用。 19

MemoVerse 的“动态”最好由 Three.js 做：

```
悬浮
翻转
旋转
景深
粒子
光照
卡片展开
票根折叠
```

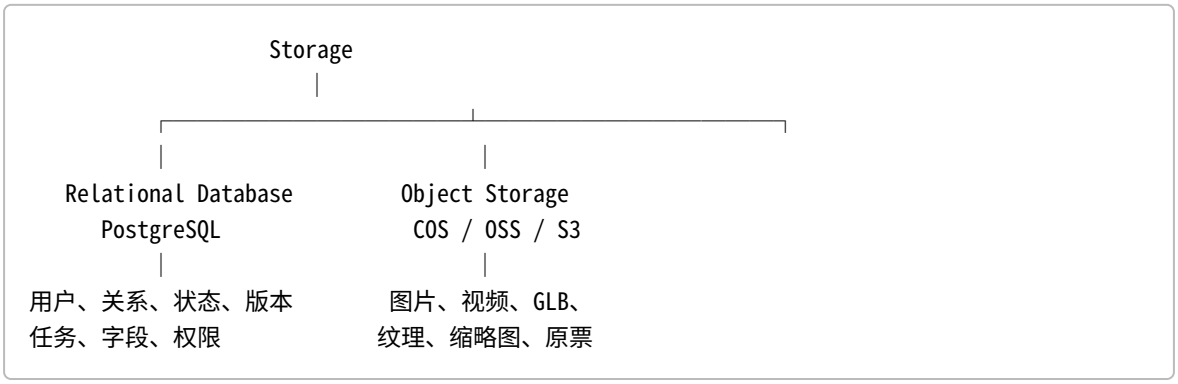
这些不要求模型有骨骼。

对象存储：COS、OSS、S3 是同一架构位置

你之前问“是不是 S3 就是对象存储”，答案是：

是。

S3、COS、OSS 都属于这里的：



Amazon 官方将 S3 定义为 object storage；对象由数据和 metadata 组成并存放在 bucket 中，同时提供访问控制、Lifecycle、Versioning 等机制。 [20]

腾讯 COS 支持预签名 URL 上传和下载，因此可以让客户端获得**有时间限制、权限限制的上传能力**，而不是把永久云密钥放进 App。 [21]

阿里 OSS 的 Browser SDK 文档更直接要求客户端环境采用 STS 临时凭据，明确不建议把账户 AccessKey 直接暴露到浏览器。 [22]

对你们的一个月项目：

方案	什么时候选	MemoVerse 需要的能力	评价
腾讯 COS	后端/服务器就在腾讯云或团队熟悉腾讯生态	私有 Bucket、临时上传、CORS、生命周期	**推荐候选**
阿里 OSS	团队已有阿里云资源	STS、Browser/Android SDK、Multipart	同样可行
AWS S3	AWS 资源或国际部署	IAM、Presigned URL、Lifecycle、Versioning	技术最标准化之一

不要同时用三家。

课程没有任何理由因为“COS + OSS + S3”而加分。

推荐对象键：

```
assets/
  originals/
    {asset_id}.jpg

  thumbnails/
    {asset_id}_small.webp

  videos/
    {asset_id}.mp4

  models/
    {asset_id}/model.glb
    {asset_id}/texture.png

  jobs/
    {job_id}/...
```

数据库只存：

```
asset_id
owner_id
```

```
object_key
mime_type
size
sha256
width
height
duration
local_state
cloud_state
created_at
```

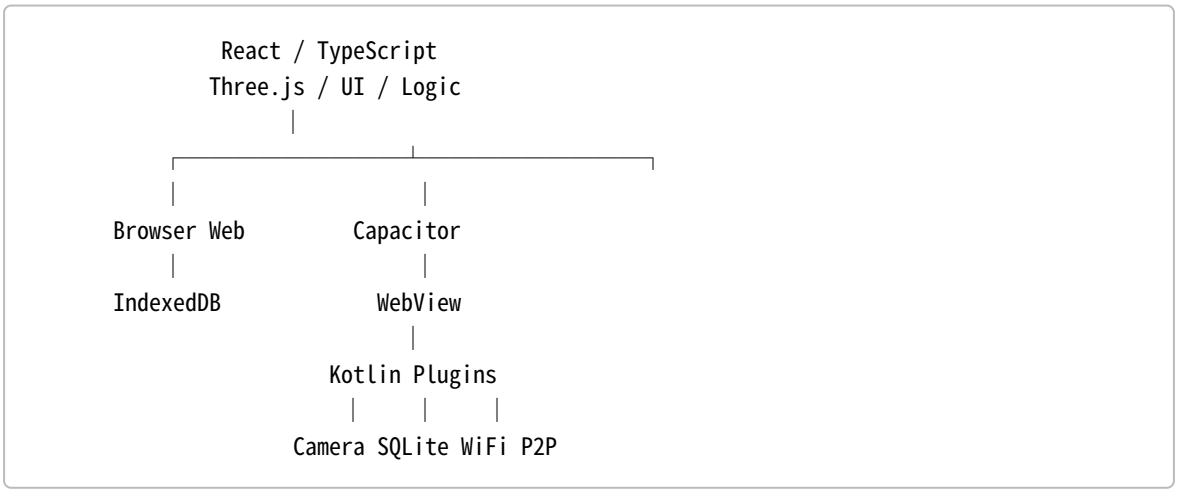
不要存大 Binary。

临时 Worker 文件应进入 `jobs/` 前缀，并设置生命周期自动清理；最终收藏资产才进入长期路径。S3 官方支持 Lifecycle 自动迁移或过期对象，COS/OSS 也提供对应对象生命周期机制。 [20]

Capacitor 的角色已经很明确

Capacitor 官方定位是 **cross-platform native runtime for Web Apps**：使用现代 Web 技术构建界面，同时保留访问 iOS/Android 原生 SDK 的能力；其 Plugin API 可以把 Android Java/Kotlin 代码暴露给 JavaScript。 [23]

所以 MemoVerse 很适合：



也就是说 Capacitor **不是“一键把网站变成完整原生 App”**。

真正的关键原生能力：

```
WiFiDirectPlugin.kt
LocalDatabasePlugin.kt    （若不用成熟 SQLite 插件）
FileTransferPlugin.kt
```

Capacitor 官方 Android Plugin Guide 本身就是让开发者通过 Java/Kotlin 接 Android SDK，然后以插件方法和事件暴露给 Web 层。 [24]

这与你们“Web + Android，而不是维护两套 UI”的项目非常匹配。

推荐系统架构、数据模型与协议

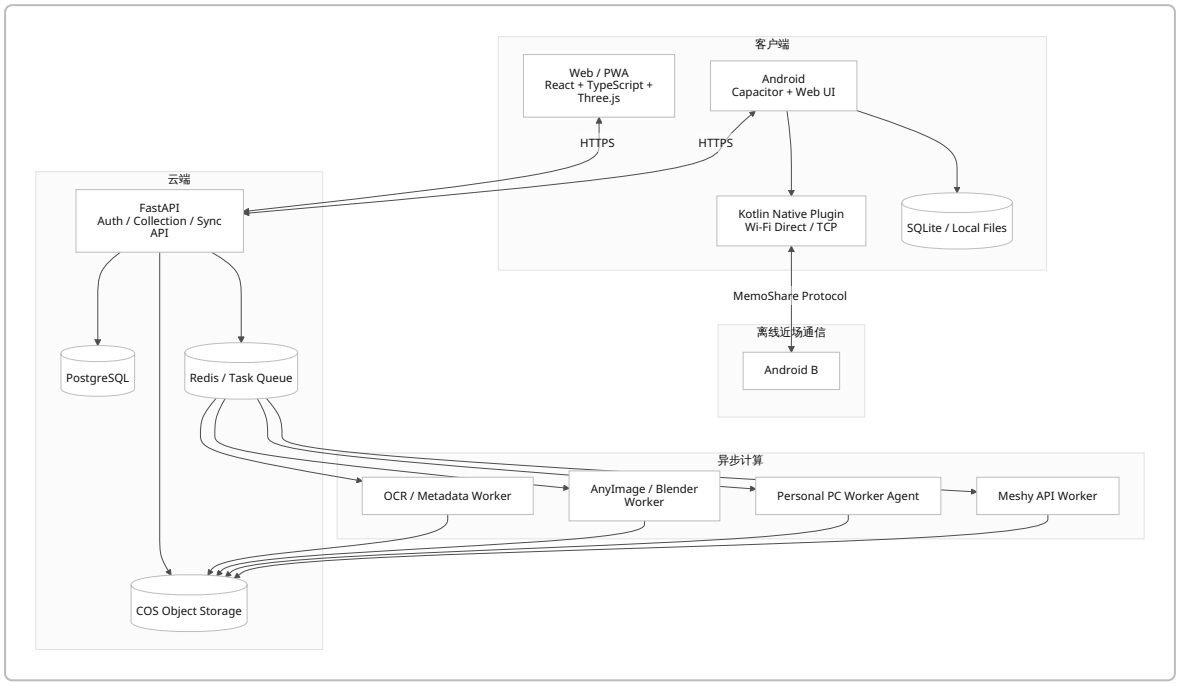
我建议不要做一堆微服务。

四个人、一个月，最适合的是：

Modular Monolith + Asynchronous Workers + Object Storage + Native P2P Adapter。

FastAPI 自身适合简单的响应后 BackgroundTasks，但其官方文档也明确区分了更重的后台计算场景；对于 Blender、OCR 批处理、3D 任务，应交给独立任务 Worker。Celery 本身就是分布式任务队列。 25

推荐总架构如下：



这张图里有一个非常重要的架构原则：

```
Control Plane → Backend / PostgreSQL
Data Plane    → COS / P2P Transfer
Compute Plane → Worker
```

例如上传一段 200 MB 视频：

错误：

```
Android
   ↓ 200MB
FastAPI
```

```
    ↓ 200MB
COS
```

更合适：

```
Android
    │
    ├──── GET upload authorization
    ▼
FastAPI

Android ──────────────直接上传──────────────→ COS

Android
    │
    └──── POST asset/complete
        ↓
      FastAPI
```

腾讯 COS 和其他主流对象存储都为这种临时授权/签名上传模式提供能力。 26

五个核心业务对象必须保留

你提出的：

```
Subject
Experience
Snapshot
CollectionEntry
Asset
```

非常适合，而且**不应该再合并成一张“Ticket”表**。

这是整个产品设计最关键的一点。

Object	含义	例子
Subject	客观“是什么”	《星际穿越》、某奶茶店、某商品、G1234
Experience	用户真实经历	“我 2026-10-01 在澳门看了这部电影”
Snapshot	某一次被固定下来的纪念品表示	海报 + 推荐语 + 票面 + 选定字段
CollectionEntry	某用户和这个纪念品的关系	OWNED / RECEIVED / WANT_TO_WATCH / FAVORITE
Asset	实际文件	JPEG、MP4、GLB、texture

为什么这很重要？

假设：

```
哲政看过电影 A
      ↓
将电影纪念卡分享给朋友 B
```

朋友 B 收到之后：

```
Subject = 电影 A                ✅
Snapshot = 哲政分享的卡片        ✅
CollectionEntry = Received      ✅

Experience = “B 看过电影 A”      ❌
```

如果以后 B 真去看了：

```
B 创建 Experience
      ↓
关联同一个 Subject
      ↓
同时可以关联：
“因为哲政 10 月 13 日推荐给我”
```

于是 MemoVerse 形成了一个非常有趣的产品关系：

```
哲政 Experience
       |
       ▼
Shared Snapshot
       |
       ▼
B Received Collection
       |
       ▼
B Want-to-watch
       |
       ▼
B Later Experience
```

这比普通票根 App 有产品意义得多。

Snapshot 为什么应该近似 Immutable

假设你今天分享：

> “这家奶茶很好喝！”

朋友收到。

一个月后你把自己记录改成：

> “第二次去觉得一般。”

如果接收方那张卡自动变成第二句话，那么：

```
分享的到底是“当时的记忆”
还是“你数据库当前版本”？
```

就混乱了。

所以建议：

```
Experience
   可编辑

        ↓ Publish / Share

Snapshot v1
   Immutable

        ↓ P2P

Recipient CollectionEntry
```

之后修改：

```
Experience
   ↓
Snapshot v2
```

原来的 v1 仍然存在。

这同时极大简化离线同步冲突。

MemoShare 应用层协议

对网络课程，我建议给协议正式命名，例如：

> **MSP — MemoShare Protocol**

基于 TCP 的一个简单 Version 1：

```
Frame:
+-------------------+
| magic "MVSP"      |
+-------------------+
| version           |
+-------------------+
| message_type      |
+-------------------+
| flags             |
+-------------------+
| payload_length    |
+-------------------+
| request_id        |
+-------------------+
| payload ...       |
+-------------------+
```

控制消息可以用 JSON 或 CBOR。

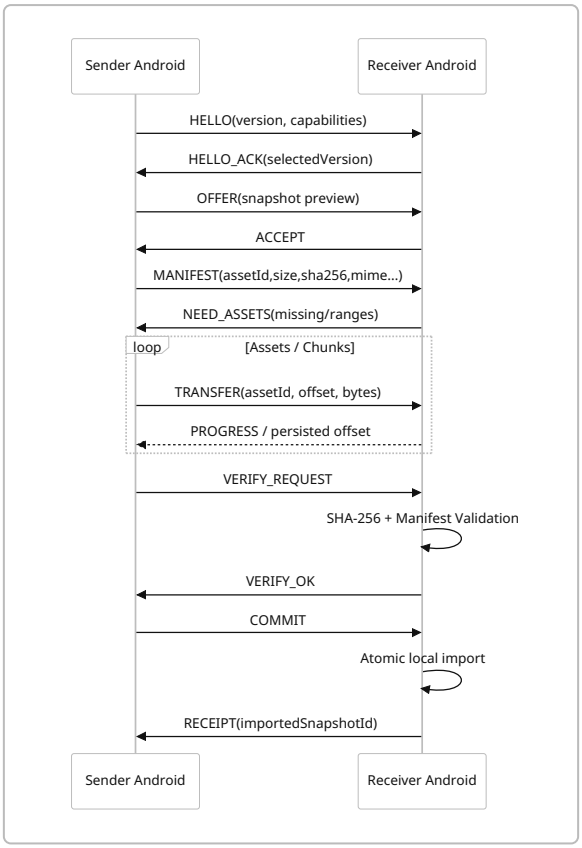
对于一个月课程项目，我反而建议控制层使用 JSON：

```
{
 "type": "OFFER",
 "protocolVersion": 1,
 "transferId": "uuid",
 "snapshotId": "uuid",
 "title": "Interstellar",
 "assetCount": 4,
 "totalBytes": 18362884
}
```

它牺牲一点编码效率，却让 Wireshark/log/debug/答辩演示非常清晰。

真正的资产内容再使用 binary frame。

完整序列：



要特别理解：

TCP 已经负责可靠、有序的字节流。

因此你们不需要在 TCP 上重新发明一个完整 TCP。

应用层 `TRANSFER / persisted offset` 的目的主要是：

```
跨连接恢复
App 被杀后的恢复
大文件部分下载状态
资产级进度
重复传输避免
```

而不是逐包解决网络丢包。

可靠传输真正需要解决以下状态：

```
CONNECTED
   ↓
NEGOTIATED
   ↓
ACCEPTED
   ↓
```

```
TRANSFERRING
   ↓
VERIFYING
   ↓
COMMITTING
   ↓
COMPLETED
```

任何一步中断：

```
Connection lost
      ↓
Transfer status = INTERRUPTED
      ↓
保留 .partial
      ↓
重新发现 / 重连
      ↓
同 transfer_id
      ↓
NEED_ASSETS(offset = ...)
      ↓
Resume
```

这才叫课程级“Reliable Transfer”。

接收端必须先 Verify，再 Commit

不能：

```
收一半视频
   ↓
直接创建正式收藏
   ↓
App 崩溃
   ↓
出现半残 CollectionEntry
```

应该：

```
Temporary Import
      ↓
所有 Mandatory Assets 完整
      ↓
SHA-256 Verify
      ↓
Database Transaction
```

```
      ↓
COMMIT
      ↓
正式 CollectionEntry
```

Optional Assets 可以稍后继续。

进一步优化体验：

```
第一阶段：
metadata + thumbnail
    ↓
朋友几乎立刻看到电影名、海报、推荐语

第二阶段：
original photo / video / GLB
    ↓
后台继续传
```

因此：

> **First Meaningful Content Time 与 Full Transfer Time 是两项不同指标。**

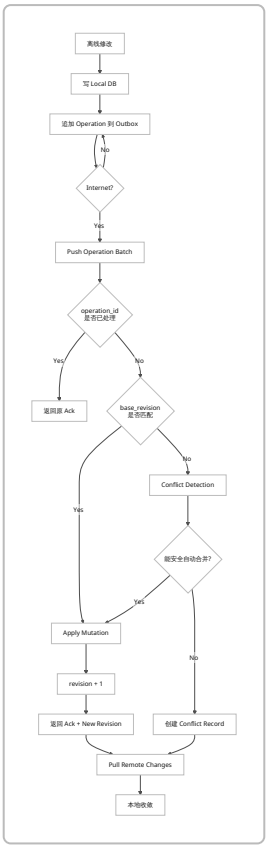
这会让 MemoVerse 的体验远好于“等 100MB 视频传完后才显示任何内容”。

离线同步不是“上传 SQLite”

Android 设备本地需要一个 Outbox / Operation Queue：

```
{
 "operationId": "uuid",
 "deviceId": "device-a",
 "entityType": "CollectionEntry",
 "entityId": "uuid",
 "operationType": "UPDATE",
 "baseRevision": 8,
 "payload": {
  "favorite": true
 }
}
```

同步流程：



这里至少要实现四件事：

Idempotency

同一个：

```
operation_id
```

客户端重试十次，服务端也只能执行一次。

Optimistic Revision

客户端：

```
base_revision = 8
```

服务端已经：

```
revision = 10
```

不能直接静默覆盖。

Conflict

例如：

```
Android Offline:
note = “好吃”

Web:
note = “一般”
```

双方恢复联网，不应单纯根据手机系统时间决定赢家。

Tombstone

删除后不是马上完全抹掉同步语义：

```
deleted = true
revision = 11
```

否则一台长时间离线的旧手机回来，可能把早已删除的记录重新上传，从而“复活”。

建议的冲突策略：

对象	推荐策略
Snapshot	Immutable，创建后原则上无编辑冲突
Asset	Immutable / content-addressed 思路
Subject	字段级更新或服务器 canonical metadata
Experience	revision 检查；同字段冲突需要用户确认
CollectionEntry.favorite	可安全使用最新确认 revision
Tags	可以做集合 merge
Delete	Tombstone 优先，不允许旧 mutation 随意复活

这部分其实比 NAT 穿透更有课程意义，因为它解决了真实的**弱连接/离线分布式状态问题**。

云端安全边界

推荐：

```
Public Internet
       |
    HTTPS
       |
   API Gateway
       |
 Auth + ACL
```

```
        |
Private PostgreSQL
```

对象存储：

```
Private Bucket
        |
        ├──── Backend 生成短期上传授权
        |
        └──── Backend 生成短期下载授权
```

不要：

```
Android APK
   ↓
写死 COS SecretKey      ❌
```

腾讯 COS 支持 Pre-Signed URL；阿里 OSS 官方 Browser SDK 也明确推荐客户端使用 STS 临时凭据而不是暴露账户 AccessKey；S3 默认对象亦为私有并通过 IAM / Bucket Policy 控制访问。 27

P2P 安全则建议分两级。

如果用 Nearby：

> Nearby 官方已经提供加密连接与双方接受流程，可以在此基础上再做 MemoShare 的用户级确认。 9

如果走原生 Wi-Fi Direct + TCP：

```
发现设备
   ↓
A 与 B 用户确认
   ↓
一次性 Pairing Secret / QR / Short Authentication String
   ↓
建立受认证的 Session
   ↓
再传输 MSP
```

密码学不要自己“设计一个算法”。使用成熟系统密码学能力，并把重点放在 Session Authentication、Replay/duplicate protection 和协议状态上。

难度论证、竞品与产品改进

1.30 是否真的站得住

我会把这个项目分成三种版本。

版本 A：集成型 Demo

```
React
+ OCR API
+ Meshy API
+ COS
+ 分享链接
```

我的判断：

> **不应该自信声称 1.30。**

技术很多，但主要是 SDK/API Integration。

版本 B：普通离线分享 App

```
Android
+ Nearby Connections
+ 图片分享
+ 云同步
```

判断：

> 有网络内容，但“Nearby 帮你做了很多底层工作”，协议与可靠性如果很浅，1.30 论据仍然一般。

Google Nearby 本来就已经抽象了发现、连接、底层链路以及 File/Stream payload，所以仅仅“成功调用 SDK 发一张图片”并不能充分证明你们自己解决了复杂网络问题。 9

版本 C：MemoVerse 目标版

```
Wi-Fi Direct Discovery
        +
Native TCP Socket
        +
Custom MemoShare Protocol
        +
Metadata-first Transfer
        +
Resume / Verify / Commit
        +
```

```
Idempotent Import
        +
Offline Operation Queue
        +
Conflict Resolution
        +
Cloud Convergence
```

判断：

> **这已经具备非常合理的 1.30 课程难度论据。**

再叠加：

```
ML Kit
AnyImage / TripoSR
Meshy
Three.js
COS
Worker
Capacitor Native Bridge
```

就是很完整的综合项目。

关键是答辩不能把技术栈念成：

> “我们用了 React、FastAPI、Redis、PostgreSQL、COS、Three.js、Meshy……”

而应该说：

> “我们识别了四个核心网络问题，并分别设计解决方案。”

例如：

网络问题	MemoVerse 解决方式
无基础设施下怎样找到另一台手机	Wi-Fi Direct Discovery
找到以后怎样表达“我要分享哪张纪念卡”	MSP OFFER / ACCEPT
大文件一半断开怎么办	Persisted offset + resume
文件收到但损坏怎么办	Manifest + SHA-256 Verify
重发导致收到两张一样的怎么办	transfer/package idempotency
收到一半 App 崩了怎么办	Verify → Atomic Commit
两台设备离线修改后怎么办	Operation queue + revision
一台设备删除、另一台很久没上线怎么办	Tombstone

网络问题	MemoVerse 解决方式
云端大文件怎么安全上传	Temporary object-storage authorization

这张表才是“1.30 的证据”。

市场竞争到底有多接近

截至本次检索，至少有两个非常直接的参照。

PocketStubs

Google Play 当前说明已经包括：

```
电影/电视剧记录
实体电影票扫描
自动加入观影历史
评分/评论
好友/社交
AI Journey Art
```

因此，“扫描电影票 → 数字化 → 视觉化收藏”已经明确存在。 28

TicketShelf

其公开描述已经更接近 MemoVerse：

```
Concert
Sports
Exhibition
Movie
Travel
AI-assisted ticket scan
ticket photos
memories
merchandise
venue map
monthly / yearly recap
shared experience
```

所以“跨类型票根生活记录”也并不是空白。 29

本次检索没有确认到一个成熟开源项目能够直接提供：

```
OCR memorabilia
+
结构化生活收藏
```

```
+
3D
+
Android 离线 P2P
+
自定义可靠传输
+
离线云同步
```

因此目前没有一个明显的“成熟开源 MemoVerse，可以直接 fork 后改 UI”的方案。这不是证明互联网上绝对没有，而是本次检索未确认到可靠、成熟、功能高度重叠的仓库。

实际更合理的路线是：

> **复用成熟组件，而不是寻找一个不存在或质量不明的完整票根系统。**

即：

```
Capacitor          → Web / Native bridge
ML Kit             → Offline OCR
Wi-Fi Direct       → P2P transport
COS                → Media storage
AnyImage/TripoSR   → Fast visual 3D
Meshy              → Detailed 3D
Three.js           → Presentation
Celery             → Workers
```

MemoVerse 的创新定位应该改变

不要写：

> “创新点：OCR 将纸质票根数字化。”

竞争产品已经做了。 6

也不要写：

> “创新点：票根收藏。”

同样不成立。

建议正式定义为：

> **MemoVerse 是一个 local-first、multimodal digital memorabilia system。它将现实生活中的凭证、照片和视频转化为可视化纪念品，并支持在无互联网环境下将结构化纪念品直接递交给附近朋友；接收者得到的是一份可继续收藏、打卡和延伸的 recommendation snapshot，而不是普通图片文件。**

真正的产品创新可以集中在下面几个点。

“Received → Experienced”

朋友分享：

```
《F1》电影
“真的很好看”
```

你收到：

```
Received
↓
Want to Watch
```

三天后你去看：

```
Experienced
↓
“由 XXX 于 10 月 13 日推荐”
```

于是一个纪念品产生**人与人之间的记忆传播路径**。

这比单纯的 P2P 文件传输更有产品意义。

Memory Card，而不是数据库详情页

正面：

```
┌──────────────────────┐
│                      │
│     MOVIE POSTER     │
│                      │
│   Interstellar       │
│   2026.10.12         │
│                      │
└──────────────────────┘
```

翻转：

```
┌──────────────────────┐
│ 原始电影票照片         │
│ 澳门 Galaxy Cinema    │
│                      │
│ “IMAX 重映，真的值得。” │
```

```
│                          │
│ Shared by 哲政           │
└──────────────────────────┘
```

Three.js 的价值应该首先服务于这种“物件感”。

Progressive Enrichment

而不是每次：

```
拍照
↓
等 OCR
↓
等内容搜索
↓
等 Meshy
↓
等上传
↓
终于能保存
```

应该：

```
拍照
↓
立即有收藏
↓
OCR enrich
↓
poster enrich
↓
3D enrich
↓
AI enrich
```

用户永远不会因为 AI 服务挂掉而“无法记下一段记忆”。

Privacy-aware Sharing

票根经常包含：

```
订单号
二维码
座位
行程
姓名
```

```
时间
地点
```

分享前应该有：

```
Share:
☑ 标题
☑ 海报
☑ 推荐语
☑ 日期
☐ 原始票据
☐ 具体座位
☐ 原始二维码
```

甚至可以：

```
OCR 检测疑似 QR / Order ID
        ↓
默认建议不分享
```

这既是产品质量，也是安全设计。

3D 应该是“纪念物表达”，不是 AI 炫技

不同类型可以拥有不同视觉模板：

```
Movie
→ floating poster card

Train
→ 立体车票 / 路线轨迹

Milk Tea
→ sticker balloon / cup badge

Restaurant
→ Polaroid + depth effect

Travel
→ postcard / map card
```

这比每张照片都强制调用 Meshy 生成一个奇怪 3D 模型更统一。

交付风险、优先级与验收测试

四个人一个月，即使大量使用 AI，最大的风险仍然不是“代码敲不完”，而是：

同时尝试太多未经验证的技术，导致每条链都只完成 70%。

AI 可以把一天的 CRUD 压缩成几小时，但不能消除：

```
两台真机为什么连不上
某厂商 ROM 为什么权限行为不同
Blender 为什么 headless 卡住
Meshy 返回的模型为什么贴图方向错
离线同步为什么出现幽灵记录
```

这些都需要真实测试和状态推理。

风险矩阵

风险	概率	影响	为什么危险	建议缓解
Wi-Fi Direct 真机碎片化	高	高	权限、厂商 ROM、设备支持存在差异	前 48h 用至少 3 个设备组合 Spike；失败就切 Nearby。Android 官方也要求检测 Wi-Fi Direct 支持并处理不同版本权限。 1
AnyImage 无法稳定自动化/达不到 1s	高	中高	它本质是 Blender 工具而非现成 Server API	持久 warm worker；独立 benchmark；快速 2.5D preview 做 fallback。 13
3D 本地方案 GPU 过重	中高	中高	InstantMesh/Hunyuan 等依赖 GPU 环境	TripoSR/AnyImage 先测；重模型列 P2；Meshy 作为云端后备。 30
Meshy 任务完成后资产丢失	中	高	第三方 Download URL 会过期	成功后立刻归档 COS。 18
OCR 多票种准确率不足	高	中	字体、反光、票据布局差异	OCR 只生成候选；必须允许人工纠正；ML Kit bundled 作为离线增强。 12
离线同步出现重复/复活/覆盖	中高	高	Distributed state 是最容易隐藏 Bug 的部分	immutable Snapshot + operation_id + revision + tombstone
Capacitor ↔ Kotlin 桥复杂度低估	中	中	网络状态是异步事件而非普通 RPC	插件接口保持很窄，业务协议留在共享 TS/Kotlin 单一一侧；Capacitor 官方支持异步 plugin calls/events。 24
范围爆炸	**很高**	**最高**	OCR、社交、3D、iOS、MCP、P2P、推荐全部想做	固定 P0/P1/P2；任何新功能必须替换，而不是累加
对象存储永久密钥泄露	中	高	APK/Web 前端可被检查	只发临时签名/STS，不将永久 Secret 放客户端。 31
版权/API 素材问题	中	中	海报、第三方模型不等于任意再分发	保存来源、授权/署名信息；开发阶段先限制为课程演示

优先级不是“谁第一周干什么”，而是系统能力的 Gate

你作为 PM 比较适合采用：

```
Vertical Slice
      ↓
可以运行
      ↓
可以失败
      ↓
失败可恢复
      ↓
体验优化
      ↓
高级视觉
```

而不是：

```
A 做前端一个月
B 做后端一个月
C 做 Android 一个月
D 做 AI 一个月
第 30 天第一次合并          ❌
```

建议优先级如下。

P0 — 没有这些，就不叫 MemoVerse

```
Android 拍照 / 导入
       ↓
Local DB + Local Asset
       ↓
手动 / OCR metadata
       ↓
Digital Collectible 2D Card
       ↓
Wi-Fi Direct P2P
       ↓
HELLO/OFFER/ACCEPT/MANIFEST
       ↓
Asset Transfer
       ↓
VERIFY/COMMIT
       ↓
Receiver Offline View
       ↓
Cloud Sync
```

```
        ↓
Web View
```

这条 Vertical Slice 一旦跑通，项目已经“活了”。

P1 — 支撑 1.30 的核心

```
Transfer Resume
Idempotent Re-import
SHA-256 Verify
Atomic Commit
Operation Queue
revision
Conflict
Tombstone
Metadata-first transfer
ML Kit bundled OCR
```

P1 — 展示增强

```
AnyImage Worker
或者 TripoSR Worker
Meshy API + COS archival
Three.js 3D collectible
```

3D 虽然很吸睛，但从课程目标来看应排在网络可靠性之后。

P2 — 时间允许才做

```
Nearby second transport
高级自动分类
大量票种 parser
公共社交 feed
MCP Server
iOS
ICE/STUN/TURN
复杂 AI Image Generation
Advanced 3D animation
```

尤其是：

```
iOS
MCP
NAT Traversal
```

绝不应该影响 Android 离线分享和同步完成度。

最应该在开发一开始做的五个 Spike

不是一周一个，而是尽快同时验证这些未知数。

Connectivity Spike

```
Android A
   ↓ Wi-Fi Direct discovery
Android B
   ↓
TCP connection
   ↓
发送 100MB 测试文件
```

测：

```
发现率
连接率
连接耗时
实际吞吐
中断行为
重新建立行为
权限体验
```

Android 官方已经证明 API 模型成立，但具体手机组合仍然属于你们需要实机验证的部分。 [1]

Capacitor Native Bridge Spike

```
React Button
   ↓
Capacitor
   ↓
Kotlin discoverPeers()
   ↓
event: peerFound
   ↓
React 显示设备
```

只要这条成立，整个前端架构风险会下降很多。Capacitor 官方 Plugin 模型支持 Kotlin 与 JavaScript 间的方法调用、权限和事件。 [24]

OCR Spike

拿真实样本：

```
10 电影票
10 高铁票
10 奶茶贴纸
10 菜单/小票
```

分别测：

```
raw text accuracy
title extraction
date
location
price
```

不要只拿一张清晰打印纸演示。

3D Spike

同十张图跑：

```
AnyImage
TripoSR
Meshy
```

记录：

```
Cold latency
Warm latency
GPU/VRAM
模型大小
视觉评分
GLB Three.js compatibility
失败率
```

只有测完，才能决定谁是：

```
Fast 3D
Detailed 3D
Fallback
```

COS Spike

```
Android/Web
  ↓ request signed upload
Backend
```

```
  ↓
signed URL
  ↓
Client direct upload
  ↓
Asset Complete
  ↓
private download
```

跑通后，大部分媒体基础设施就稳定了。COS 官方已经支持预签名 URL 上传下载，因此这条路线技术上有成熟基础。[21]

推荐最终验收测试矩阵

ID	测试	操作	必须看到的结果
T01	完全离线创建	Android 飞行模式/无互联网，拍照	本地成功保存、App 重启仍存在
T02	离线 OCR	无网下使用 bundled ML Kit	能识别则生成候选；失败也不影响收藏创建
T03	P2P Discovery	两机都无互联网	能发现对方并显示用户可确认设备
T04	正常 Share	分享含图像的 Snapshot	HELLO→…→COMMIT 完整完成
T05	Metadata-first	分享含大视频	B 先看到 title/poster/description，媒体继续传
T06	中断恢复	传 50% 后关闭 Wi-Fi	Partial 被保留；重连后续传，不从零开始
T07	Integrity	人工破坏 Partial	VERIFY 失败，不允许 Commit
T08	Duplicate	同 transfer_id 再传一次	不产生重复收藏
T09	Crash Recovery	TRANSFER 中杀 App	重启后能识别 interrupted transfer
T10	Sender Delete	B 收到后 A 删除自己的收藏	B 已导入副本仍正常存在
T11	Offline→Cloud	B 收到时无网，随后联网	云端最终出现 B 的 CollectionEntry
T12	Web Convergence	B Android 同步完成	Web 页面显示相同收藏
T13	Conflict	Android 离线改 note；Web 同时改 note	系统检测并按照定义规则处理
T14	Delete Tombstone	Web 删除，旧 Android 隔天上线	旧数据不会无条件“复活”
T15	Meshy Archive	生成成功后归档	后续不依赖 Meshy 临时 URL 仍能展示
T16	Permission Denied	拒绝 Nearby/Wi-Fi 权限	App 不崩溃，并给出正确恢复操作
T17	Large Asset	视频 + GLB + 图片	协议正确处理进度/取消/恢复

ID	测试	操作	必须看到的结果
T18	Low Storage	接收端存储不足	Transfer 失败但数据库不出现完整收藏
T19	Protocol Version	A=v1，B=不兼容版本	HELLO 阶段拒绝，而不是传到一半崩溃
T20	AnyImage Benchmark	Cold/Warm 各跑多组样本	得到真实 p50/p95，不再凭感觉宣称 1 秒

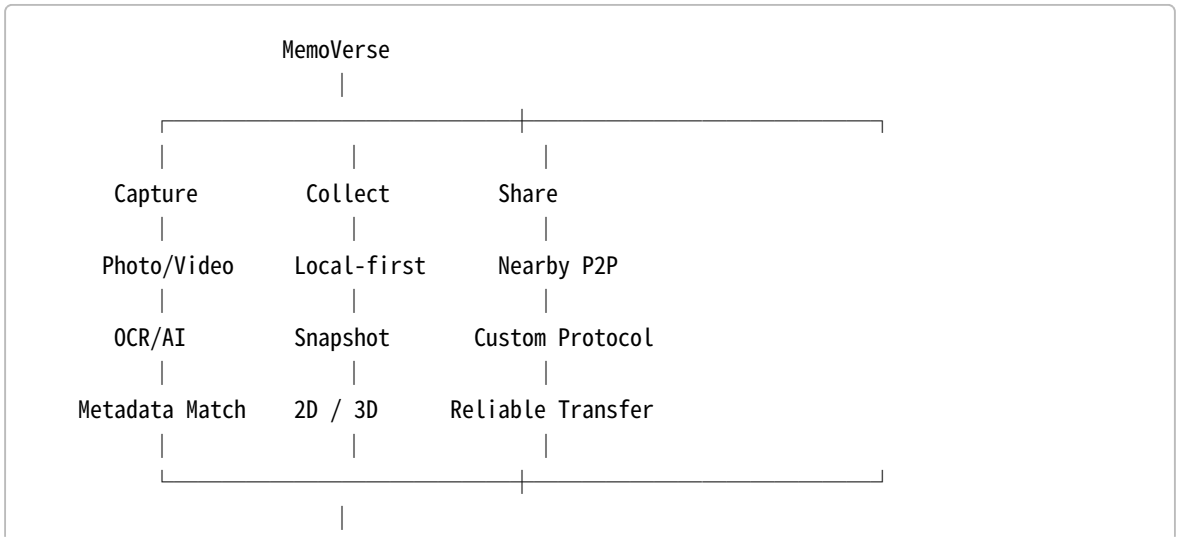
建议网络验收指标写成**项目目标值**，而不是现在伪造结果：

Metric	建议验收目标
Discovery Success Rate	⩾95%，在所支持的测试机型组合上
Normal Transfer Integrity	100% 测试样本 hash 一致
Resume Integrity	100% 测试成功，无文件损坏
Duplicate Import	0
Interrupted Transfer Data Loss	不超过最后未持久化 chunk
Metadata First Visible	建连接受后尽量控制在约 1–2 秒
Offline→Cloud Convergence	稳定互联网恢复后约数秒到十余秒级目标
Conflict silently overwritten	0
AnyImage warm latency	**以实测决定是否达到 1 秒**
Meshy archival	100% 成功任务归档至自有对象存储

这些是**工程验收目标**，不是已经测得的性能数据。

最值得保留和最值得砍掉的功能

最终建议的项目重心可以浓缩成下面这张图：



```
                    Sync
                      |
            Operation / Conflict
                      |
                Web + Cloud
```

应该牢牢保留：

```
照片/视频记录
OCR + 人工确认
Subject / Experience / Snapshot 区分
本地收藏
无互联网 Android P2P
自定义协议
断点恢复
完整性验证
离线云同步
冲突处理
Web + Android
COS
至少一种 3D 视觉增强
```

很有价值但可以降级：

```
ML Kit Offline OCR
AnyImage
TripoSR
Meshy
高级海报匹配
地图/年度回顾
```

应该坚决放到 Future Work：

```
iOS 正式版
NAT Traversal
ICE/STUN/TURN
公共社交网络
所有票种完美识别
区块链/NFT
复杂真实性认证
多人 P2P 网络
MCP Server 作为主要业务入口
任意物体自动骨骼动画
```

最终我对项目的判断是：

从技术可行性看：高。 Android 官方 Wi-Fi Direct 原生 Socket、Nearby Connections、ML Kit、Capacitor、成熟对象存储以及多种现成 3D 路线都已经存在，不需要从零发明底层平台。 32

从一个月交付看：中高风险但可控。 风险主要来自 Wi-Fi Direct 真机行为、离线同步和 3D Worker 自动化，而不是普通 CRUD。最重要的是前 48 小时先验证未知技术，然后持续维护一个随时可运行的端到端版本。

从网络编程课程契合度看：高，但有条件。 条件是不能把 Nearby/COS/Meshy 的 SDK 调用当作网络深度本身。真正要展示的是 P2P discovery、raw socket/custom protocol、resumable transfer、integrity、atomic import、offline operation log 和 conflict resolution。

从 1.30 难度论证看：目标版具备充分合理性，但不能保证老师最终给分。 如果可靠传输与离线同步只存在于 PPT，项目会下降成普通集成应用；如果它们能现场故障演示并恢复，网络技术深度会非常明显。

从创意看：基础票根数字化约为中等创新，完整 MemoVerse 概念则明显更强。 PocketStubs 和 TicketShelf 已经证明票根扫描、生活档案、AI 辅助和社交记录并非空白；MemoVerse 应主动放弃“票根 OCR 首创”的叙事，把创新集中到 **“生活纪念品 + 视觉物件化 + local-first + nearby handoff + recommendation-to-experience”**。 6

而从产品设计上，我认为目前最值得继续迭代的核心不是再增加一个技术，而是回答这个问题：

> **“为什么朋友不用微信发一张图片，而要用 MemoVerse 把一个纪念品递给我？”**

目前最有潜力的答案已经出现：

> 因为他递给我的不是一张 JPEG，而是一份**有名字、有来源、有故事、有视觉表达、可以离线接收、可以收藏、可以标记‘想去/想看’，并且未来可以变成我自己真实 Experience 的数字纪念品**。

一旦下一轮把这个业务闭环进一步固定下来，MemoVerse 就可以从“很丰富的学生技术项目”收敛成一个**业务逻辑清楚、网络难度扎实、展示效果强，而且一个月内存在完成路径的完整开题项目**。

---

1 32 Wi-Fi Direct (peer-to-peer or P2P) overview | Connectivity | Android Developers
https://developer.android.com/develop/connectivity/wifi/wifip2p

2 7 9 Overview | Nearby Connections | Google for Developers
https://developers.google.com/nearby/connections/overview

3 RFC 8445: Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal | RFC Editor
https://www.rfc-editor.org/info/rfc8445/

4 13 GitHub - OmooLab/AnyImage: Fast image-guided structure building tools for Blender · GitHub
https://github.com/OmooLab/AnyImage

5 30 GitHub - VAST-AI-Research/TripoSR: TripoSR: Fast 3D Object Reconstruction from a Single Image · GitHub
https://github.com/VAST-AI-Research/TripoSR

6 28 PocketStubs: Movie TV Tracker, - Apps on Google Play
https://play.google.com/store/apps/details?hl=en&id=com.pocketstubs.app

8 Bluetooth overview | Connectivity | Android Developers
https://developer.android.com/develop/connectivity/bluetooth

10 Multipeer Connectivity | Apple Developer Documentation
https://developer.apple.com/documentation/multipeerconnectivity?utm_source=chatgpt.com

11 Get started | Nearby Connections | Google for Developers
https://developers.google.com/nearby/connections/swift/get-started

12 Recognize text in images with ML Kit on Android | Google for Developers
https://developers.google.com/ml-kit/vision/text-recognition/v2/android

14 GitHub - TencentARC/InstantMesh: InstantMesh: Efficient 3D Mesh Generation from a Single Image with Sparse-view Large Reconstruction Models • GitHub
https://github.com/TencentARC/InstantMesh

15 GitHub - Tencent-Hunyuan/Hunyuan3D-2: High-Resolution 3D Assets Generation with Large Scale Hunyuan3D Diffusion Models. • GitHub
https://github.com/Tencent-Hunyuan/Hunyuan3D-2

16 17 Meshy API Dokumentasi: Rujukan & Panduan
https://docs.meshy.ai/ms/api?utm_source=chatgpt.com

18 Meshy MCP Server — 3D Generation in Local AI Agents
https://www.meshy.ai/mcp?utm_source=chatgpt.com

19 Rigging API - Meshy Docs
https://docs.meshy.ai/en/api/rigging?utm_source=chatgpt.com

20 What is Amazon S3? - Amazon Simple Storage Service
https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

21 26 27 31 Generating Pre-Signed URLs
https://www.tencentcloud.com/document/product/436/37680?utm_source=chatgpt.com

22 Object Storage Service:Install the OSS Browser.js SDK
https://www.alibabacloud.com/help/en/oss/developer-reference/installation?utm_source=chatgpt.com

23 Capacitor - Cross-platform Native Runtime for Web Apps | Capacitor Documentation
https://capacitorjs.com/docs

24 Capacitor Android Plugin Guide | Capacitor Documentation
https://capacitorjs.com/docs/plugins/android

25 Background Tasks
https://fastapi.tiangolo.com/tutorial/background-tasks/?utm_source=chatgpt.com

29 TicketShelf: Concert Stub Log - Apps on Google Play
https://play.google.com/store/apps/details?hl=en&id=com.defineyou.ticketshelf